Secure Login System: Task Report

Name: Maheshwaran S | **Task No.:** 4 | **Organization:** Thiranex | **Due Date:** 07 Oct 2026

1. Introduction

Authentication is the first line of defense for any web application. Weak login systems expose user accounts to password theft, SQL injection, and session hijacking. This task involved developing a secure login web application that follows modern security practices: hashed password storage, input validation, protection against SQL injection, safe session management, and optional Two-Factor Authentication (2FA).

2. Objectives

- Build user registration and login using securely hashed passwords (bcrypt or Argon2).
- Apply input validation and protect the database from SQL injection.
- Implement session management with a working logout feature.
- (Optional) Add Two-Factor Authentication for an extra layer of security.

3. Technologies Used

(Edit this table to match what you actually used.)

Component	Technology
Backend	Python (Flask)
Database	SQLite / MySQL
Password hashing	bcrypt
Frontend	HTML, CSS, JavaScript
2FA (optional)	TOTP via pyotp + authenticator app
Tools	VS Code, Postman, browser DevTools

4. System Design

The application has four main modules: Registration, Login, Session/Dashboard, and Logout, with an optional 2FA verification step after login.

Workflow:

1. The user submits the registration form. Inputs are validated and the password is hashed before being stored.
2. The user logs in with credentials. The server fetches the stored hash using a parameterized query and verifies the password.
3. If 2FA is enabled, the user enters a one-time code from their authenticator app.
4. On success, a session is created and the user reaches the protected dashboard.
5. Logout destroys the session and redirects to the login page.

Database schema (users table):

Field	Type	Description
id	INTEGER (PK)	Unique user ID
username	TEXT (UNIQUE)	Login name
email	TEXT (UNIQUE)	User email
password_hash	TEXT	bcrypt/Argon2 hash (never plain text)
totp_secret	TEXT (nullable)	Secret key for 2FA

5. Implementation

5.1 Password Hashing

Passwords are never stored in plain text. Each password is hashed with bcrypt, which adds a unique random salt and is deliberately slow, making brute-force attacks costly. During login, the entered password is checked against the stored hash using the library's verify function.

5.2 Input Validation

All inputs are validated on both the client and server side:

- Username and email are checked for allowed characters and valid format.
- Passwords must meet a minimum length and complexity (uppercase, lowercase, digit, special character).
- Empty or oversized fields are rejected.
- Error messages are generic ("Invalid username or password") so attackers cannot tell which field was wrong.

5.3 SQL Injection Protection

All database queries use parameterized (prepared) statements instead of string concatenation. User input is treated strictly as data, never as executable SQL. For example, an input such as `' OR '1'='1` is stored or compared as plain text and does not bypass authentication.

5.4 Session Management

- A session is created only after successful authentication.
- Session cookies are set with the `HttpOnly` and `SameSite` flags (and `Secure` when running on HTTPS).
- Protected pages check for a valid session and redirect unauthenticated users to the login page.
- The logout feature clears the session data completely.
- Sessions expire after a period of inactivity.

5.5 Two-Factor Authentication (Optional)

A time-based one-time password (TOTP) is used. A secret key is generated for the user and shown as a QR code to scan with an authenticator app. After the correct password, the user must enter the current 6-digit code, which changes every 30 seconds. Access is granted only if both factors are valid.

6. Testing and Results

Test Case	Input	Expected Result	Result
Valid registration	Valid details	Account created, password hashed	Pass
Duplicate user	Existing username	Error shown	Pass
Weak password	"12345"	Rejected	Pass
Valid login	Correct credentials	Dashboard opens	Pass
Wrong password	Incorrect password	Generic error message	Pass
SQL injection	`' OR '1'='1`	Login denied	Pass
Direct URL access	/dashboard without login	Redirect to login	Pass
Logout	Click logout	Session destroyed	Pass

Test Case	Input	Expected Result	Result
2FA	Wrong or expired code	Access denied	Pass

The database showed only hashed values in the password field, confirming that credentials are not exposed even if the database is leaked.

7. Challenges and Solutions

- **Handling sessions securely:** Resolved by using secure cookie flags and server-side checks on every protected route.
- **Preventing SQL injection:** Resolved by replacing all concatenated queries with parameterized queries.
- **Setting up 2FA:** Resolved by using a standard TOTP library and testing with an authenticator app.

8. Conclusion

The Secure Login System meets the task requirements. Hashed passwords, input validation, SQL injection protection, and session management with logout significantly reduce unauthorized access and protect user accounts from common attacks. The optional 2FA adds another layer of defense even if a password is compromised.

9. Future Enhancements

- Account lockout or rate limiting after repeated failed logins
- Email verification and password reset via secure tokens
- CAPTCHA to block automated attacks
- CSRF tokens on all forms
- Login activity logging and alerts

10. References

- OWASP Authentication Cheat Sheet
- OWASP SQL Injection Prevention Cheat Sheet
- bcrypt and pyotp library documentation
- Task tutorial provided on the Thiranex student portal